

**LAPORAN PRAKTIKUM
IMPLEMENTASI PERANGKAT LUNAK**

**MODUL 4
DESIGN PATTERN & OBJECT COMPOSING**

DISUSUN OLEH :

AMMAR BAGAS FATHURRAHMAN WANTORO 2350081008



**PROGRAM STUDI INFORMATIKA
FAKULTAS SAINS DAN INFORMATIKA
UNIVERSITAS JENDERAL ACHMAD YANI
TAHUN 2025**

DAFTAR ISI

DAFTAR ISI	i
DAFTAR GAMBAR	iii
BAB I. HASIL PRAKTIKUM	4
I.1 Student.java	4
I.1.A. Source Code	4
I.1.B. Analisa	4
I.2 StudentView.java	5
I.2.A. Source code	5
I.2.B. Analisa	5
I.3 StudentController.java	6
I.3.A. Source code	6
I.3.B. Analisa	7
I.4 MVCPatternDemo.java	8
I.4.A. Source code	8
I.4.B. Hasil	8
I.4.C. Analisa	8
BAB II. TUGAS PRAKTIKUM	10
II.1 MVCPatternDemo	10
II.1.A. Source code	10
II.1.B. Hasil	11
II.1.C. Analisa	11
II.2 Course.java	12
II.2.A. Source code	12
II.2.B. Analisa	13
II.3 CourseView.java	13

II.3.A. Source code	13
II.3.B. Analisa	14
II.4 CourseController.java	14
II.4.A. Source code	14
II.4.B. Analisa	15
II.5 Barang.java	16
II.5.A. Source code	16
II.5.B. Analisa	17
II.6 KasirView.java	18
II.6.A. Source code	18
II.6.B. Analisa	20
II.7 KasirController.java	20
II.7.A. Source code	20
II.7.B. Analisa	22
II.8 Main.java	22
II.8.A. Source code	22
II.8.B. Hasil	23
II.8.C. Analisa	23
BAB III. KESIMPULAN	25

DAFTAR GAMBAR

gambar 1. 1 Hasil MVC	8
gambar 2. 1 Hasil MVC Course	11
gambar 2. 2 Hasil program kasir	23

BAB I. HASIL PRAKTIKUM

I.1 Student.java

I.1.A. Source Code

```
package Latihan;

public class Student {
    private String rollNo;
    private String name;

    public String getRollNo() {
        return rollNo;
    }

    public void setRollNo(String rollNo) {
        this.rollNo = rollNo;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

I.1.B. Analisa

Analisis kode pada kelas Student menunjukkan penerapan konsep fundamental Enkapsulasi yang menjadi dasar pembentukan objek dalam pemrograman berorientasi objek. Atribut rollNo dan name dideklarasikan dengan akses modifier private, yang bertujuan untuk menyembunyikan state internal objek dari akses langsung pihak luar, sementara metode public (getter dan setter) disediakan sebagai gerbang akses yang terkontrol. Struktur ini menjadikan kelas Student sebagai POJO (Plain Old Java Object) atau Java Bean yang memenuhi standar object composing, di mana integritas data terjaga karena setiap perubahan nilai harus melalui metode

yang telah ditentukan, memungkinkan penyisipan validasi di masa depan tanpa merusak kode klien yang menggunakannya.

Dalam konteks Design Pattern, kelas ini berperan sebagai Transfer Object (TO) atau Value Object (VO), yang merupakan komponen utama dalam pola arsitektur seperti Transfer Object Pattern atau sebagai Model dalam pola MVC (Model-View-Controller). Tanggung jawab tunggal kelas ini hanyalah untuk menampung data dan memindahkannya antar lapisan (layer) aplikasi, misalnya dari database ke client, tanpa dibebani oleh logika bisnis yang kompleks. Kesederhanaan ini sangat krusial dalam object composing, karena memungkinkan objek Student untuk dengan mudah dikomposisikan atau digunakan kembali oleh berbagai layanan (services) atau pola desain lain (seperti DAO Pattern) tanpa menciptakan ketergantungan yang rumit (low coupling).

I.2 StudentView.java

I.2.A. Source code

A. Class Car

```
package Latihan;

public class StudentView {
    public void printStudentDetails(String studentName,
                                    String studentRollNo) {
        System.out.println("Student: ");
        System.out.println("Name: " + studentName);
        System.out.println("Roll No: " + studentRollNo);
    }
}
```

I.2.B. Analisa

Analisis kode pada kelas StudentView menunjukkan implementasi komponen View dalam pola arsitektur Model-View-Controller (MVC). Dalam konteks design pattern, kelas ini memiliki tanggung jawab tunggal (Single Responsibility) untuk menangani lapisan presentasi atau visualisasi data kepada pengguna. Dengan memisahkan logika tampilan ini dari kelas data (Student), desain ini menegaskan pemisahan kepentingan (separation of concerns), di mana perubahan pada cara data

ditampilkan (misalnya format teks di konsol) tidak akan mempengaruhi struktur data itu sendiri atau logika bisnis aplikasi.

Dari sisi object composing, metode `printStudentDetails` bertindak sebagai antarmuka pasif yang menerima data mentah (nama dan nomor induk) yang biasanya dikirimkan oleh komponen Controller. Pendekatan ini menciptakan loose coupling (ketergantungan rendah), karena `StudentView` tidak perlu mengetahui logika internal objek `Student` atau bagaimana data tersebut diproses; ia hanya bertugas memformat dan mencetak output. Struktur modular ini sangat penting dalam pengembangan perangkat lunak karena memudahkan pemeliharaan dan skalabilitas, di mana antarmuka pengguna dapat dimodifikasi atau diganti sepenuhnya tanpa merusak integritas backend sistem.

I.3 StudentController.java

I.3.A. Source code

```
package Latihan;

public class StudentController {
    private Student model;
    private StudentView view;

    public StudentController(Student model, StudentView view) {
        this.model = model;
        this.view = view;
    }

    public void setStudentName(String name) {
        model.setName(name);
    }

    public String getStudentName() {
        return model.getName();
    }

    public void setStudentRollNo(String rollNo) {
        model.setRollNo(rollNo);
    }
}
```

```
public String getStudentRollNo() {  
    return model.getRollNo();  
}  
  
public void updateView() {  
    view.printStudentDetails(model.getName(),  
        model.getRollNo());  
}  
}
```

I.3.B. Analisa

Analisis kode pada kelas `StudentController` menunjukkan peran vitalnya sebagai Controller dalam pola arsitektur MVC (Model-View-Controller), yang bertindak sebagai jembatan atau mediator antara lapisan data (`Student`) dan lapisan presentasi (`StudentView`). Dalam konteks Object Composing, kelas ini mendemonstrasikan penggunaan komposisi objek (association) di mana ia "memiliki" referensi ke objek Model dan View sebagai atribut privatnya, alih-alih menggunakan pewarisan. Konstruktor kelas ini menerapkan prinsip Dependency Injection, yang memungkinkan Controller untuk merakit dan mengelola interaksi antara Model dan View tanpa membuat keduanya saling bergantung (decoupled), sehingga struktur aplikasi menjadi lebih modular dan terorganisir.

Secara fungsional, `StudentController` bertanggung jawab penuh atas logika kendali dan aliran data aplikasi. Metode-metode setter dan getter di dalamnya digunakan untuk memanipulasi data pada Model, sementara metode `updateView()` bertugas menyinkronkan data tersebut ke tampilan. Pada metode `updateView()`, Controller mengambil data terkini dari Model dan menyuntikkannya ke dalam metode `printStudentDetails` milik View. Mekanisme ini memastikan bahwa View tidak pernah mengakses Model secara langsung, melainkan selalu melalui Controller, menjaga pemisahan tanggung jawab (Separation of Concerns) yang merupakan tujuan utama dari pola desain ini.

..

I.4 MVCPatternDemo.java

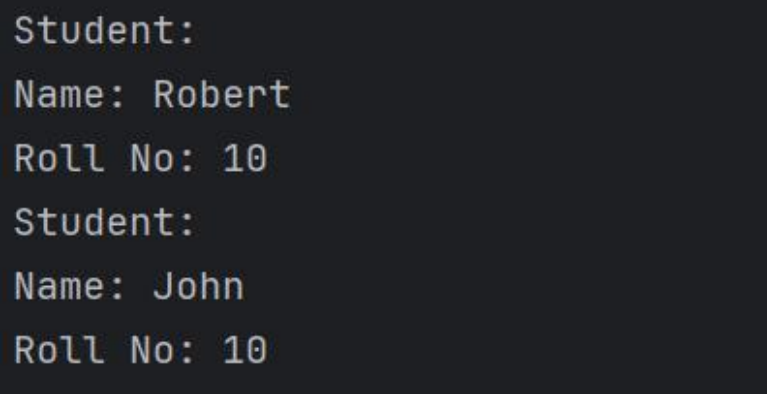
I.4.A. Source code

```
package Latihan;

public class MVCPatternDemo {
    public static void main(String[] args) {
        Student model = retrieveStudentFromDatabase();
        StudentView view = new StudentView();
        StudentController controller = new StudentController(model, view);
        controller.updateView();
        controller.setStudentName("John");
        controller.updateView();
    }

    private static Student retrieveStudentFromDatabase()
    {
        Student student = new Student();
        student.setName("Robert");
        student.setRollNo("10");
        return student;
    }
}
```

I.4.B. Hasil



```
Student:
Name: Robert
Roll No: 10
Student:
Name: John
Roll No: 10
```

gambar 1. 1 Hasil MVC

I.4.C. Analisa

Analisis pada kelas MVCPatternDemo menunjukkan perannya sebagai Client atau titik eksekusi utama yang mendemonstrasikan implementasi pola MVC (Model-View-Controller) secara utuh. Di dalam metode main, terjadi proses Object Composing yang krusial, di mana komponen-komponen yang terpisah—Student

(Model) yang disimulasikan diambil dari database dan StudentView (View)—diinstansiasi secara independen lalu dirakit ke dalam StudentController. Konstruksi ini memperlihatkan kekuatan dependency injection, di mana Controller tidak membuat dependensinya sendiri, melainkan menerima objek Model dan View dari luar, menciptakan sistem yang fleksibel di mana komponen dapat diganti atau diuji secara terisolasi (decoupling).

Secara fungsional, kelas ini memvalidasi alur kerja MVC dengan melakukan manipulasi data sepenuhnya melalui Controller, tanpa berinteraksi langsung dengan Model atau View untuk memperbarui tampilan. Kode ini mensimulasikan siklus hidup aplikasi yang dinamis: dimulai dengan menampilkan data awal ("Robert"), kemudian klien memicu perubahan logika bisnis (setStudentName menjadi "John"), dan akhirnya meminta pembaruan tampilan (updateView). Mekanisme ini membuktikan keberhasilan enkapsulasi logika, di mana Controller bertindak sebagai manajer pusat yang memastikan bahwa setiap perubahan pada data (state) direfleksikan dengan tepat pada antarmuka pengguna tanpa klien perlu mengetahui kompleksitas internal sistem.

BAB II. TUGAS PRAKTIKUM

II.1 MVCPatternDemo

II.1.A. Source code

```
package Tugas;

public class MVCPatternDemo {
    public static void main(String[] args) {

        Course model = retrieveCourseFromDatabase();

        CourseView view = new CourseView();
        CourseController controller = new CourseController(model,
view);
        controller.updateView();

        controller.setCourseName("Python");
        System.out.println("\nAfter updating, Course Details are as
follows");
        controller.updateView();
    }

    private static Course retrieveCourseFromDatabase() {
        Course course = new Course();
        course.setName("Java");
        course.setId("01");

        course.setCategory("Programming");
        return course;
    }
}
```

II.1.B. Hasil

```
5\prak-ip1\p4\untitled\out\production\untitled" Tugas
.MVCPatternDemo
Course Details:
Name: Java
Course ID: 01
Course Category: Programming

After updating, Course Details are as follows
Course Details:
Name: Python
Course ID: 01
Course Category: Programming
```

gambar 2.1 Hasil MVC Course

II.1.C. Analisa

Analisis pada kelas MVCPatternDemo menunjukkan fungsinya sebagai Client atau driver utama yang bertugas merakit seluruh komponen arsitektur MVC (Model-View-Controller). Kode ini mendemonstrasikan penerapan Object Composing yang efektif, di mana objek Course (Model) yang mensimulasikan pengambilan data dari basis data dan CourseView (View) diinstansiasi secara terpisah sebagai entitas yang independen. Kedua objek ini kemudian disatukan atau "dikomposisikan" ke dalam CourseController melalui konstruktor, menciptakan struktur ketergantungan yang longgar (loose coupling) di mana Controller bertindak sebagai pengendali pusat yang menghubungkan data dengan tampilannya tanpa membuat Model dan View saling bergantung secara langsung.

Secara operasional, kelas ini memvalidasi alur kerja dan sinkronisasi data dalam pola MVC. Dimulai dengan menampilkan status awal data kursus ("Java"), kode kemudian mensimulasikan perubahan bisnis dengan memanggil `controller.setCourseName("Python")`. Poin penting dari analisis ini adalah bahwa perubahan data dan pembaruan tampilan (`updateView`) dilakukan sepenuhnya melalui perantara Controller, bukan dengan memanipulasi Model atau View secara langsung. Hal ini membuktikan keberhasilan prinsip Separation of Concerns, di mana logika aplikasi dan presentasi data terisolasi dengan baik, memudahkan pemeliharaan dan pengujian sistem di masa depan.

II.2 Course.java

II.2.A. Source code

```
package Tugas;

public class Course {
    private String name;
    private String id;
    private String category;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getId() {
        return id;
    }

    public void setId(String id) {
        this.id = id;
    }

    public String getCategory() {
        return category;
    }

    public void setCategory(String category) {
        this.category = category;
    }
}
```

II.2.B. Analisa

Analisis pada kelas Course menunjukkan penerapan prinsip dasar pemrograman berorientasi objek, yaitu Enkapsulasi. Atribut name, id, dan category dideklarasikan sebagai private untuk menyembunyikan state internal objek dari akses luar secara langsung, menjaga integritas data agar tidak dimanipulasi sembarangan. Sebagai gantinya, akses diberikan melalui metode public (getter dan setter) yang bertindak sebagai gerbang terkontrol. Struktur ini menjadikan kelas Course sebagai POJO (Plain Old Java Object) atau Java Bean yang ideal untuk object composing, karena ia menyediakan kerangka data standar yang mudah diinstansiasi dan dikelola oleh komponen lain dalam sistem.

Dalam konteks pola desain MVC (Model-View-Controller), kelas ini berperan murni sebagai Model. Tanggung jawab tunggalnya (Single Responsibility) adalah merepresentasikan entitas bisnis (dalam hal ini, sebuah Mata Kuliah) dan menampung data yang relevan. Kelas ini didesain untuk tidak memiliki ketergantungan apa pun terhadap logika tampilan (View) atau logika kendali (Controller), sehingga menciptakan pemisahan kepentingan (Separation of Concerns) yang jelas. Keterpisahan ini memungkinkan objek Course digunakan kembali di berbagai bagian aplikasi atau bahkan dikirimkan antar layer sebagai Transfer Object tanpa membawa beban logika yang tidak perlu.

II.3 CourseView.java

II.3.A. Source code

```
package Tugas;

public class CourseView {
    public void printCourseDetails(String CourseName, String
CourseId, String CourseCategory) {
        System.out.println("Course Details: ");
        System.out.println("Name: " + CourseName);
        System.out.println("Course ID: " + CourseId);
        System.out.println("Course Category: " + CourseCategory);
    }
}
```

II.3.B. Analisa

Analisis pada kelas `CourseView` memperlihatkan peran spesifiknya sebagai komponen `View` dalam arsitektur Model-View-Controller (MVC). Kelas ini mengemban tanggung jawab tunggal (Single Responsibility) untuk menangani aspek presentasi, yaitu bagaimana informasi disampaikan kepada pengguna akhir. Dengan mengisolasi logika tampilan—dalam hal ini pencetakan ke konsol—ke dalam kelas tersendiri, kode ini menegakkan prinsip Separation of Concerns. Artinya, struktur data internal atau logika pemrosesan data tidak tercampur dengan kode antarmuka, menjaga kebersihan kode dan memastikan bahwa perubahan pada cara data ditampilkan tidak akan mempengaruhi logika bisnis.

Dari perspektif Object Composing, metode `printCourseDetails` didesain sebagai antarmuka pasif yang hanya menerima data mentah (seperti nama, ID, dan kategori) yang "didorong" oleh komponen eksternal, yaitu `Controller`. Desain ini menciptakan loose coupling (ketergantungan rendah) yang sangat baik karena `CourseView` tidak perlu mengetahui keberadaan atau struktur internal dari objek Model (`Course`); ia hanya peduli pada parameter string yang diterima. Sifat modular ini sangat menguntungkan dalam pengembangan perangkat lunak, karena memungkinkan pengembang untuk memodifikasi atau mengganti implementasi tampilan sepenuhnya (misalnya dari konsol ke antarmuka grafis/GUI) tanpa risiko merusak logika bisnis di bagian lain sistem.

II.4 CourseController.java

II.4.A. Source code

```
package Tugas;

public class CourseController {
    private Course model;
    private CourseView view;

    public CourseController(Course model, CourseView
        view) {
        this.model = model;
        this.view = view;
    }
}
```

```

    public void setCourseName(String name) {
        model.setName(name);
    }

    public String getCourseName() {
        return model.getName();
    }

    public void setCourseId(String id) {
        model.setId(id);
    }

    public String getCourseId() {
        return model.getId();
    }

    public void setCourseCategory(String category) {
        model.setCategory(category);
    }

    public String getCourseCategory() {
        return model.getCategory();
    }

    public void updateView() {
        view.printCourseDetails(model.getName(),
                                model.getId(), model.getCategory());
    }
}

```

II.4.B. Analisa

Analisis pada kelas CourseController menunjukkan perannya sebagai Controller yang menjadi pusat orkestrasi dalam pola arsitektur MVC. Dalam konteks Object Composing, kelas ini menerapkan hubungan asosiasi di mana ia "memiliki" referensi ke objek Course (Model) dan CourseView (View) sebagai atribut privatnya, yang disuntikkan melalui mekanisme Dependency Injection pada konstruktor. Desain ini menempatkan Controller sebagai mediator yang memisahkan Model dan View, memastikan bahwa kedua komponen tersebut tetap

decoupled (tidak bergantung satu sama lain secara langsung) dan hanya berinteraksi melalui perantara ini.

Secara fungsional, kelas ini bertanggung jawab penuh atas kendali logika dan sinkronisasi data. Metode-metode setter di dalamnya berfungsi untuk meneruskan input atau perubahan state ke Model, sedangkan metode `updateView()` bertugas mengambil data terkini dari Model dan menyuplainya ke View untuk ditampilkan. Mekanisme ini menegaskan prinsip Separation of Concerns, di mana Controller menjamin bahwa View tidak pernah mengakses properti Model secara langsung, melainkan menerima data matang yang siap tampil, sehingga integritas logika bisnis dan fleksibilitas tampilan dapat terjaga dengan baik.

II.5 Barang.java

II.5.A. Source code

```
package Tugas;

public class Barang {
    private String nama;
    private int harga;
    private int stok;

    public Barang(String nama, int harga, int stok) {
        this.nama = nama;
        this.harga = harga;
        this.stok = stok;
    }

    public String getNama() {
        return nama;
    }

    public int getHarga() {
        return harga;
    }

    public int getStok() {
        return stok;
    }
}
```

```

    public void setName(String nama) {
        this.nama = nama;
    }

    public void setHarga(int harga) {
        this.harga = harga;
    }

    public void setStok(int stok) {
        this.stok = stok;
    }

    public boolean kurangiStok(int jumlah) {
        if (stok >= jumlah) {
            stok -= jumlah;
            return true;
        }
        return false;
    }

    public int hitungTotal(int qty) {
        return harga * qty;
    }
}

```

II.5.B. Analisa

Analisis pada kelas Barang menunjukkan penerapan konsep Enkapsulasi yang kuat sebagai fondasi utama dalam Object Composing. Atribut nama, harga, dan stok bersifat private, melindungi state internal objek dari manipulasi langsung yang tidak valid. Kelas ini dirancang sebagai Domain Entity atau Model yang merepresentasikan benda nyata dalam sistem. Tidak seperti Data Transfer Object (DTO) murni yang hanya berisi getter dan setter, kelas ini mulai mengintegrasikan perilaku (behavior) ke dalam data melalui konstruktor yang memastikan objek selalu diinisialisasi dalam keadaan valid, menjadikannya komponen yang mandiri dan siap dikomposisikan ke dalam sistem yang lebih besar (seperti sistem Transaksi atau Inventaris).

Secara fungsional, kelas ini menerapkan prinsip Rich Domain Model di mana logika bisnis dasar diletakkan langsung di dalam objek pemilik data. Metode `kurangiStok` dan `hitungTotal` adalah contoh nyata enkapsulasi logika bisnis; objek `Barang` bertanggung jawab penuh memvalidasi ketersediaan stok sebelum melakukan pengurangan dan melakukan kalkulasi harganya sendiri. Pendekatan ini mencegah terjadinya Anemic Domain Model dan mengurangi kompleksitas pada layer eksternal (seperti Controller), karena aturan validasi dan operasi aritmatika melekat erat pada objeknya, memastikan integritas data tetap terjaga di mana pun objek ini digunakan.

II.6 KasirView.java

II.6.A. Source code

```
package Tugas;

import java.util.Scanner;

public class KasirView {
    private Scanner scanner;

    public KasirView() {
        scanner = new Scanner(System.in);
    }

    public void tampilkanInfoBarang(Barang barang) {

        System.out.println("=====");
        System.out.println("Nama Barang : " +
            barang.getNama());
        System.out.println("Harga Barang : " +
            barang.getHarga());
        System.out.println("Stok Barang : " +
            barang.getStok());

        System.out.println("=====");
    }

    public void tampilkanHeader() {
        System.out.println("\n=== Transaksi Pembelian ===");
    }
}
```

```

    }

    public int inputJumlahBarang() {
        System.out.print("Jumlah barang : ");
        return scanner.nextInt();
    }

    public void tampilkanPembayaran(int total) {

        System.out.println("\n=====");
        System.out.println("Jumlah Bayar : " + total);

        System.out.println("=====");
    }

    public void tampilkanPesanError(String pesan) {
        System.out.println("ERROR: " + pesan);
    }

    public void tampilkanPesanSukses(String pesan) {
        System.out.println("SUCCESS: " + pesan);
    }

    public void tampilkanInfoStokBaru(Barang barang) {
        System.out.println("\n=== Info Stok Terbaru ===");
        tampilkanInfoBarang(barang);
    }

    public void closeScanner() {
        scanner.close();
    }
}

```

II.6.B. Analisa

Analisis pada kelas KasirView menunjukkan peran krusialnya sebagai komponen View dalam pola arsitektur Model-View-Controller (MVC). Kelas ini memegang tanggung jawab tunggal (Single Responsibility) untuk menangani seluruh interaksi pengguna (User Interface), baik itu menampilkan informasi produk dan struk pembayaran ke konsol maupun menangkap input jumlah pembelian. Dengan mengisolasi logika antarmuka (Input/Output) dari logika bisnis transaksi, desain ini menerapkan prinsip Separation of Concerns yang efektif. Hal ini memastikan bahwa perubahan pada estetika atau format tampilan—seperti mengubah desain struk atau pesan error—tidak akan mengintervensi atau merusak logika perhitungan stok dan harga yang berada di lapisan Model atau Controller.

Dari perspektif Object Composing, kelas ini mengenkapsulasi objek Scanner sebagai atribut privat untuk menangani input stream, memastikan pengelolaan sumber daya input dilakukan secara terpusat dan aman. Metode-metode seperti tampilkanInfoBarang mendemonstrasikan interaksi di mana View menerima objek Barang (Model) sebagai parameter semata-mata untuk keperluan visualisasi (read-only), tanpa memiliki wewenang untuk memodifikasi state objek tersebut. Struktur ini mendukung pola komunikasi yang dikendalikan penuh oleh Controller, di mana View menyediakan antarmuka pasif berisi metode feedback (sukses/error) dan input data yang siap dipanggil, menjaga alur aplikasi tetap terorganisir dan komponen-komponennya tetap tidak terikat erat (loosely coupled).

II.7 KasirController.java

II.7.A. Source code

```
package Tugas;

public class KasirController {
    private Barang barang;
    private KasirView view;

    public KasirController(Barang barang, KasirView view) {
        this.barang = barang;
        this.view = view;
    }
}
```

```

public void executeTransaction() {
    view.tampilkanInfoBarang(barang);

    processPayment();

    view.closeScanner();
}

private void processPayment() {
    view.tampilkanHeader();

    int qty = view.inputJumlahBarang();

    if (qty <= 0) {
        view.tampilkanPesanError("Jumlah barang harus lebih
dari 0!");
        return;
    }

    if (qty > barang.getStok()) {
        view.tampilkanPesanError("Stok tidak mencukupi! Stok
tersedia: " + barang.getStok());
        return;
    }

    int total = barang.hitungTotal(qty);
    barang.kurangiStok(qty);

    view.tampilkanPembayaran(total);
    view.tampilkanInfoStokBaru(barang);
    view.tampilkanPesanSukses("Transaksi berhasil!");
}
}

```

II.7.B. Analisa

Analisis pada kelas `KasirController` menunjukkan perannya sebagai `Controller` yang berfungsi sebagai otak atau pengatur alur utama dalam pola arsitektur MVC. Dalam konteks Object Composing, kelas ini menerapkan hubungan asosiasi dengan mengkomposisikan objek `Barang` (Model) dan `KasirView` (View) sebagai atribut internalnya melalui mekanisme `Dependency Injection` pada konstruktor. Desain ini menempatkan `Controller` sebagai mediator yang menjembatani komunikasi antara data dan tampilan, memastikan bahwa kedua komponen tersebut tetap loosely coupled (tidak terikat langsung). Dengan demikian, `Controller` memegang kendali penuh atas siklus hidup transaksi tanpa membebani Model dengan urusan tampilan atau sebaliknya.

Secara operasional, metode `executeTransaction` dan `processPayment` mendemonstrasikan orkestrasi logika bisnis yang terpusat. `Controller` bertugas mengambil input jumlah dari pengguna melalui View, melakukan validasi logis (seperti pengecekan stok negatif atau tidak mencukupi), dan kemudian memanggil metode bisnis pada Model (`hitungTotal` dan `kurangiStok`) untuk memproses perubahan data. Pendekatan ini menegaskan prinsip `Separation of Concerns`, di mana `Controller` tidak melakukan perhitungan harga secara internal (tugas Model) atau mencetak teks ke layar (tugas View), melainkan hanya mengarahkan "siapa yang harus melakukan apa", menjaga alur aplikasi tetap terstruktur, aman, dan mudah dipelihara.

II.8 Main.java

II.8.A. Source code

```
import Tugas.Barang;
import Tugas.KasirController;
import Tugas.KasirView;

public class Main {
    public static void main(String[] args) {

        Barang barang = new Barang("Donat", 3000, 10);

        KasirView view = new KasirView();
```

```

        KasirController controller = new KasirController(barang,
view);

        controller.executeTransaction();
    }
}

```

II.8.B. Hasil

```

=== Info Stok Terbaru ===
=====
Nama Barang : Donat
Harga Barang : 3000
Stok Barang : 5
=====
SUCCESS: Transaksi berhasil!

Process finished with exit code 0

```

gambar 2. 2 Hasil program kasir

II.8.C. Analisa

Analisis pada kelas Main menunjukkan fungsinya sebagai Composition Root atau titik awal eksekusi sistem, di mana prinsip Object Composing didemonstrasikan secara nyata. Di sini, komponen-komponen arsitektur MVC diinisialisasi sebagai entitas yang terpisah dan independen: objek Barang (Model) dibuat dengan data awal dan objek KasirView (View) disiapkan untuk antarmuka. Kedua objek ini kemudian "dirakit" atau dikomposisikan ke dalam KasirController melalui konstruktor. Pendekatan ini membuktikan bahwa sistem dibangun di atas prinsip loose coupling, di mana Model dan View tidak saling mengenal atau terikat satu sama lain sejak awal, melainkan disatukan oleh pihak luar (kelas Main) sesuai kebutuhan.

Secara teknis, kelas ini menerapkan mekanisme Dependency Injection manual untuk menghubungkan dependensi antar objek. Alih-alih membiarkan

KasirController membuat instansi Model atau View-nya sendiri (yang akan menciptakan tight coupling), kelas Main yang bertanggung jawab menyuntikkan dependensi tersebut. Setelah proses perakitan selesai, kendali alur program diserahkan sepenuhnya kepada Controller melalui metode executeTransaction(). Struktur ini secara efektif memisahkan fase konfigurasi (pembuatan dan perakitan objek) dari fase runtime (eksekusi logika bisnis), menjadikan aplikasi lebih fleksibel dan mudah diuji karena setiap komponen dapat diganti atau dimodifikasi tanpa mengubah struktur inti pengontrolnya.

BAB III. KESIMPULAN

Praktikum "Design Pattern & Object Composing" ini telah mendemonstrasikan secara komprehensif bagaimana penerapan pola arsitektur Model-View-Controller (MVC) dapat meningkatkan kualitas struktur perangkat lunak secara signifikan. Melalui studi kasus manajemen Student dan sistem transaksi Kasir, kita melihat penerapan nyata prinsip Separation of Concerns (pemisahan kepentingan). Logika bisnis dan integritas data terisolasi dengan aman di dalam Model (seperti kelas Barang dan Student), interaksi pengguna ditangani secara eksklusif oleh View, dan seluruh alur kerja diorkestrasi oleh Controller. Struktur ini mencegah terjadinya kode yang saling membelit (spaghetti code) dan memastikan setiap kelas mematuhi Single Responsibility Principle.

Lebih jauh lagi, praktikum ini memperlihatkan kekuatan dari Object Composing (komposisi objek) dibandingkan pewarisan yang kaku. Dengan menggunakan mekanisme Dependency Injection di kelas Main (sebagai Composition Root), kita merakit aplikasi dengan menyuntikkan objek Model dan View ke dalam Controller. Pendekatan ini menghasilkan sistem yang Loosely Coupled (berketergantungan rendah), di mana komponen-komponen bersifat independen dan modular. Hal ini memberikan fleksibilitas tinggi dalam pengembangan; misalnya, kita dapat mengubah antarmuka View atau mengganti sumber data Model tanpa perlu membongkar logika Controller, menjadikan aplikasi lebih mudah dipelihara (maintainable), diuji (testable), dan dikembangkan (scalable) di masa depan.